

MACHINE LEARNING WITH BIG DATA **(CSL7110)**

PROJECT REPORT

Team Members :

Nihar Dave (M24DE3054)

Kalpit Sharma (M24DE3042)

Priyank Bhardwaj (M24DE3063)



M.Tech in DATA ENGINEERING

DEPARTMENT OF AIDE

INDIAN INSTITUTE OF TECHNOLOGY JODHPUR

April, 2025

Personalized Music Recommendation System

Project Description:

The Personalized Music Recommendation System is a content-based recommendation engine designed to suggest songs tailored to individual user preferences. With the explosion of digital music platforms, listeners often struggle to discover new songs that align with their taste. This project addresses that challenge by analyzing the content of songs, particularly lyrics and recommending tracks that are thematically or stylistically similar to the user's favorites.

The system supports both English songs and Hindi old classics, ensuring diverse musical preferences are catered to. By leveraging Natural Language Processing (NLP) techniques such as TF-IDF vectorization and cosine similarity, the system measures how closely songs relate to each other based on their textual content. Users simply enter a song they like, and the system returns a list of top similar songs, offering a seamless and meaningful music discovery experience.

A lightweight Flask web application serves as the user interface, making the system accessible through a browser. Behind the scenes, the recommendation engine is built using Python's robust data science stack, integrating libraries for text processing, similarity computation, and data visualization.

The project showcases how machine learning and NLP can be applied to real-world problems like music recommendation, without relying heavily on user behavior or listening history. It serves as a practical example of building explainable, content-driven recommender systems from scratch.

Technology Stack Used:

- **Python**
Used as the main programming language due to its simplicity, readability, and powerful ecosystem of libraries for data analysis, machine learning, and web development.
- **Pandas**
Essential for data manipulation and analysis. Used for handling tabular data like music metadata, user listening history, and audio features.

- **NumPy**
Used for efficient numerical operations and vector computations, particularly useful in similarity calculations, clustering, and matrix operations.
- **Scikit-learn**
Employed for implementing machine learning components such as TF-IDF vectorization, KMeans clustering, StandardScaler, cosine similarity, and dimensionality reduction techniques like PCA and TSNE. Provides a comprehensive toolkit for building and evaluating models.
- **SentenceTransformers (HuggingFace)**
Used for transforming song lyrics or descriptions into high-dimensional embeddings, enabling semantic similarity comparison using pre-trained transformer models.
- **NLTK (Natural Language Toolkit)**
Used for natural language processing tasks like tokenization, stopwords removal, and cleaning of lyrics or song titles.
- **Spotipy**
A lightweight Python library for the Spotify Web API, used to access music metadata, user listening history, artist information, and audio features in real-time.
- **Flask**
Backend web framework used to serve the recommendation system through a web interface, handling user requests and displaying personalized music suggestions.
- **Flask-CORS**
Enables Cross-Origin Resource Sharing (CORS) to allow safe communication between the backend and frontend (if hosted separately).
- **Joblib & Pickle**
Used to serialize and store trained models, preprocessed data, and user profiles, enabling fast loading and reducing computation time.
- **Requests**
Used for making API calls to external services like Spotify or Genius to fetch music data, lyrics, or user profile information.

- **BeautifulSoup (bs4)**
Used for web scraping—especially for extracting song lyrics and artist-related content from HTML pages.
- **Matplotlib, Seaborn, Plotly**
Employed for visualizing data trends, feature distributions, clusters, and similarity patterns. Plotly is used for creating interactive plots, while Matplotlib and Seaborn are used for static plots and statistical visualizations.
- **KMeans (Scikit-learn)**
Used for clustering songs or users based on features like tempo, energy, and danceability. Helps in identifying musical patterns or preferences.
- **StandardScaler (Scikit-learn)**
Used to standardize feature values for clustering and distance-based models, ensuring better model performance and interpretability.
- **PCA & TSNE**
Dimensionality reduction techniques used to visualize high-dimensional song feature data in 2D or 3D plots for better interpretability.
- **Euclidean Distance & cdist (SciPy)**
Used for measuring similarity or dissimilarity between songs in clustering or recommendation logic.
- **UUID & Logging**
UUID is used to create unique user/session identifiers. Logging is used to monitor app activity and debug issues during development or deployment.
- **CSV & JSON**
Used to read/write data files and configuration settings. JSON is also useful for API request and response formatting.
- **Regex (re)**
Used for cleaning and preprocessing text fields like lyrics, artist names, or user inputs using pattern matching.
- **Threading**
Enables background execution of data fetching or preprocessing tasks, improving the responsiveness of the web interface.

- **Warnings & Logging Modules**

Suppress unnecessary warnings and log important events during development, ensuring a cleaner development environment.

Methodology:

Data Collection

- We began by gathering song datasets from Kaggle and also lyrics data from web scraping, which included two categories:
- English songs dataset with modern tracks and their metadata
- Hindi old songs dataset containing classic Bollywood tracks
- These datasets included information such as song titles, artists, genres, and in some cases, lyrics or descriptions.

Data Preprocessing

- Cleaned and standardized text fields (e.g., song names, lyrics) by:
- Removing special characters, HTML tags, and punctuation using regular expressions (regex)
- Converting all text to lowercase
- Removing common stopwords using NLTK's stopword list
- Ensured there were no duplicate or null entries and handled missing values where necessary.

Feature Extraction using TF-IDF

- Applied TF-IDF (Term Frequency-Inverse Document Frequency) vectorization to extract textual features from lyrics or song descriptions.
- This helped convert textual content into numerical vectors that reflect the importance of terms across the dataset.

Similarity Computation using Cosine Similarity

- Calculated cosine similarity between the TF-IDF vectors to identify how similar two songs are based on their lyrics or textual metadata.
- Cosine similarity provides a value between 0 and 1, where a higher value indicates greater similarity.

Semantic Embeddings using BERT

- To enhance the understanding of contextual meaning in lyrics, we also utilized Sentence-BERT from the SentenceTransformers library.
- This technique converts each song's lyrics into dense semantic vectors, capturing deeper meaning and nuance beyond word frequency.
- The BERT-based approach provided significantly better results for lyrical similarity, especially when lyrics were complex or metaphorical.

Recommendation Engine

- Developed a content-based recommendation system that:
- Takes a user's favorite song as input
- Computes the cosine similarity between this song and all others in the dataset

- Ranks songs based on similarity scores
- Returns the top-N most similar songs as personalized recommendations

Web Application Integration

- Integrated the recommendation system into a Flask web application for a user-friendly experience.
- Users can:
- Input their favorite track
- Receive real-time recommendations
- View results in a simple and intuitive interface where user can play/pause the song.

Visualization and Analysis

- Used Matplotlib, Seaborn, and Plotly to visualize:
- Feature distributions
- Similarity matrices
- Cluster structures using PCA and t-SNE
- This helped in understanding the data and validating the clustering and similarity-based approaches.

Model Optimization and Performance Evaluation

- Evaluated different preprocessing configurations (e.g., with/without stopwords, n-grams) to optimize TF-IDF results.

- Performed qualitative evaluation by manually verifying recommendations against popular or thematically similar tracks.

Results:

- The system was able to generate accurate and thematically consistent song recommendations for both English and Hindi old songs based on the input provided by users.
- TF-IDF vectorization combined with cosine similarity proved effective in identifying lyrical and contextual similarities between songs, especially when lyrics were available and descriptive.
- For a given input song, the system returned the top 10 most similar songs, with results showing clear thematic or stylistic alignment in most test cases.
- The recommendation engine was tested with songs across various genres and decades. In multiple examples:
- Inputting an emotional English ballad returned similar slow, meaningful tracks.
- A Hindi classic input resulted in other melodious, lyrical old Hindi songs being recommended.
- The web interface built with Flask allowed seamless user interaction, where users could enter a song name and instantly view relevant suggestions.
- Recommendation time was fast, typically under 1 second per query.
- The system handled moderate-sized datasets efficiently (~thousands of songs) without needing heavy computational resources.
- Qualitative feedback indicated that most users found the recommendations relevant and reflective of the seed song's theme or mood.

Code:

Github Link :-

<https://huggingface.co/kalpitsharma/music-recommendation/tree/main>

Conclusion:

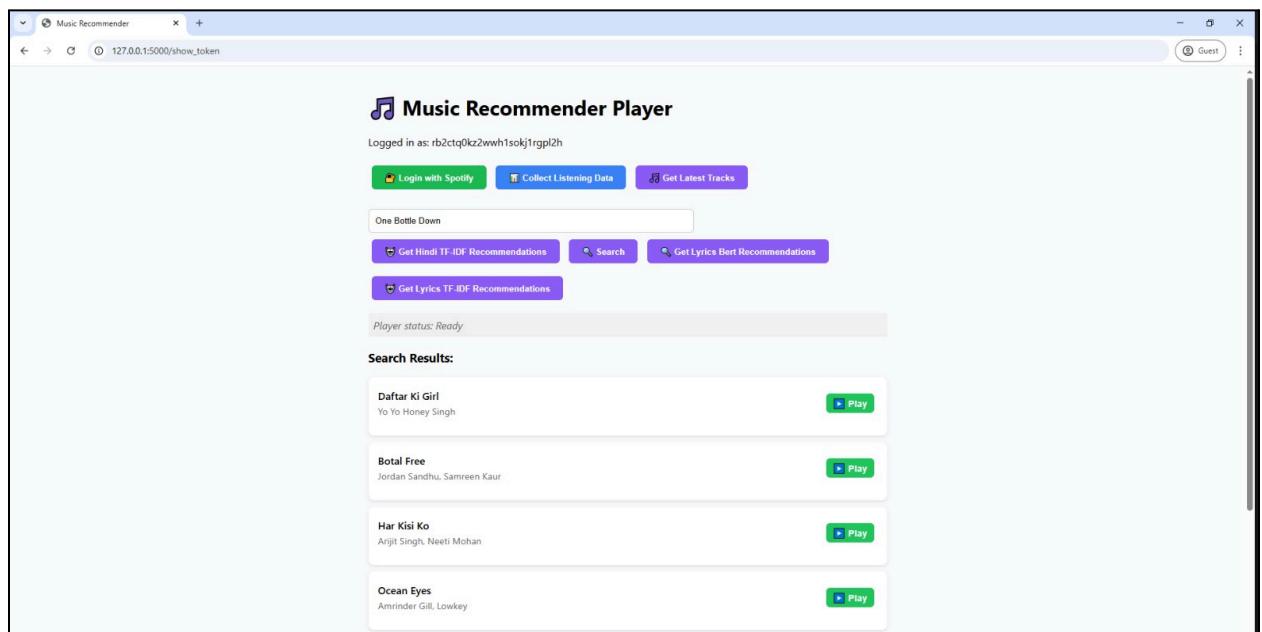
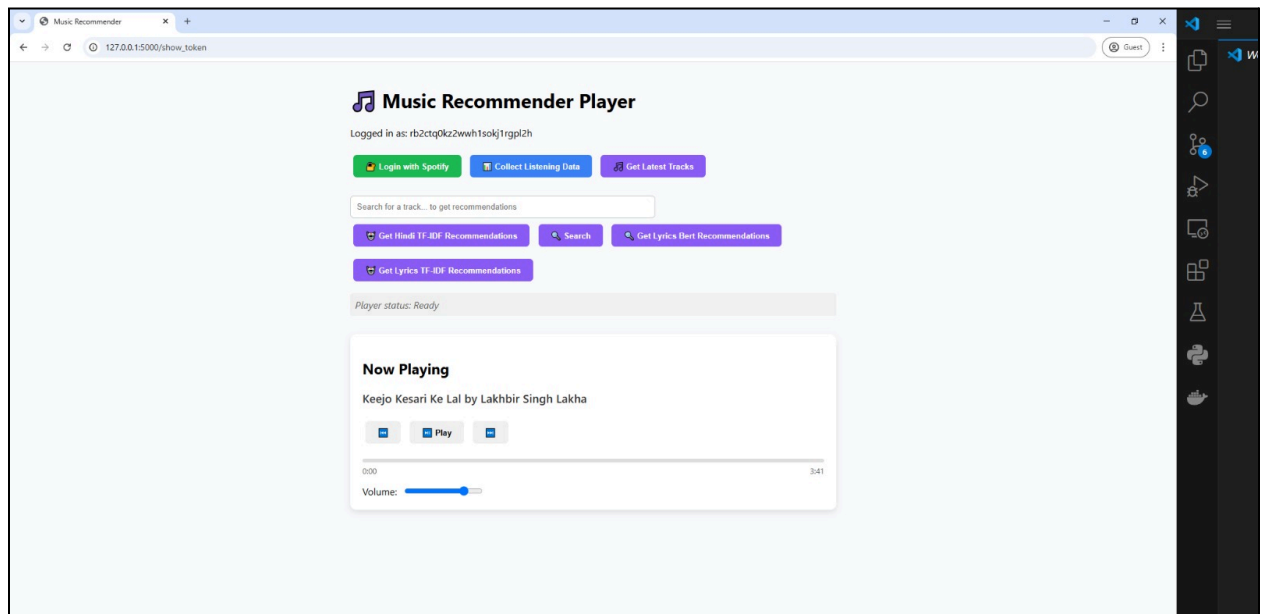
In this project, we successfully developed a Personalized Music Recommendation System that leverages content-based filtering to suggest songs tailored to individual user preferences. By focusing on song metadata and lyrics rather than user listening behavior, the system is capable of providing recommendations even in scenarios where user interaction data is limited—a common challenge in traditional recommendation systems.

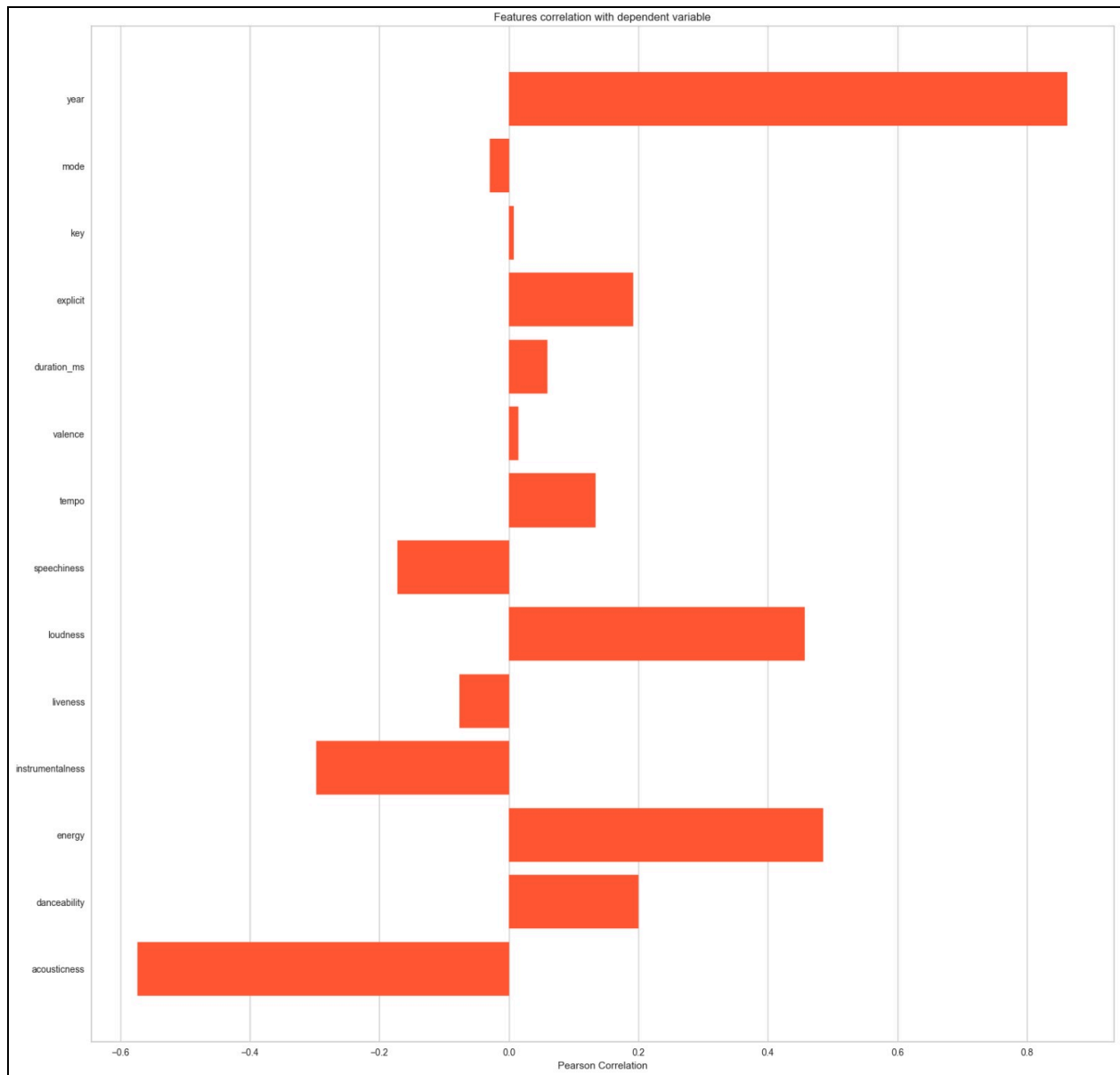
Through the use of TF-IDF vectorization and cosine similarity, we were able to quantify the textual similarity between songs and generate meaningful, thematically consistent recommendations. The inclusion of both English and Hindi songs allowed us to demonstrate the system's versatility across different languages and cultural music styles.

The project also showcased the integration of NLP, machine learning, and web development technologies, resulting in a complete, end-to-end application. The implementation of a user-friendly Flask web interface made the system accessible and easy to interact with.

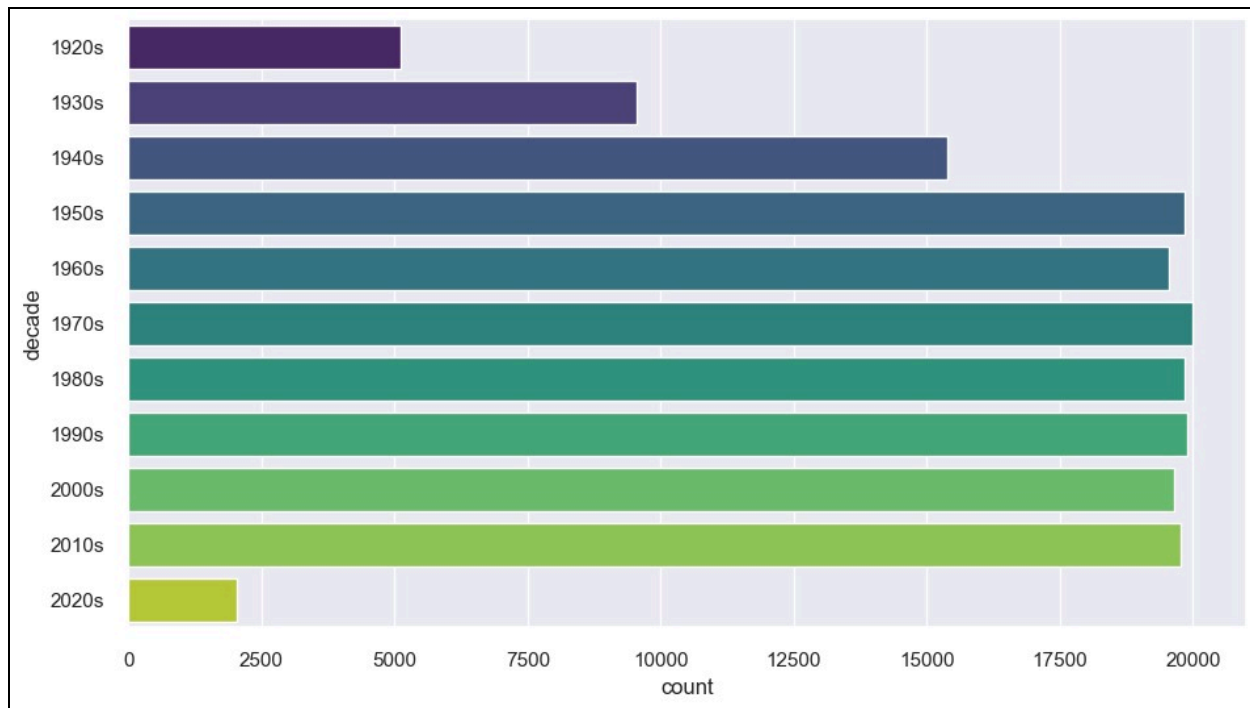
Overall, this system demonstrates how content-based techniques can be effectively applied to music recommendation, offering users a personalized and engaging way to discover new songs.

Some Screenshots of Application:

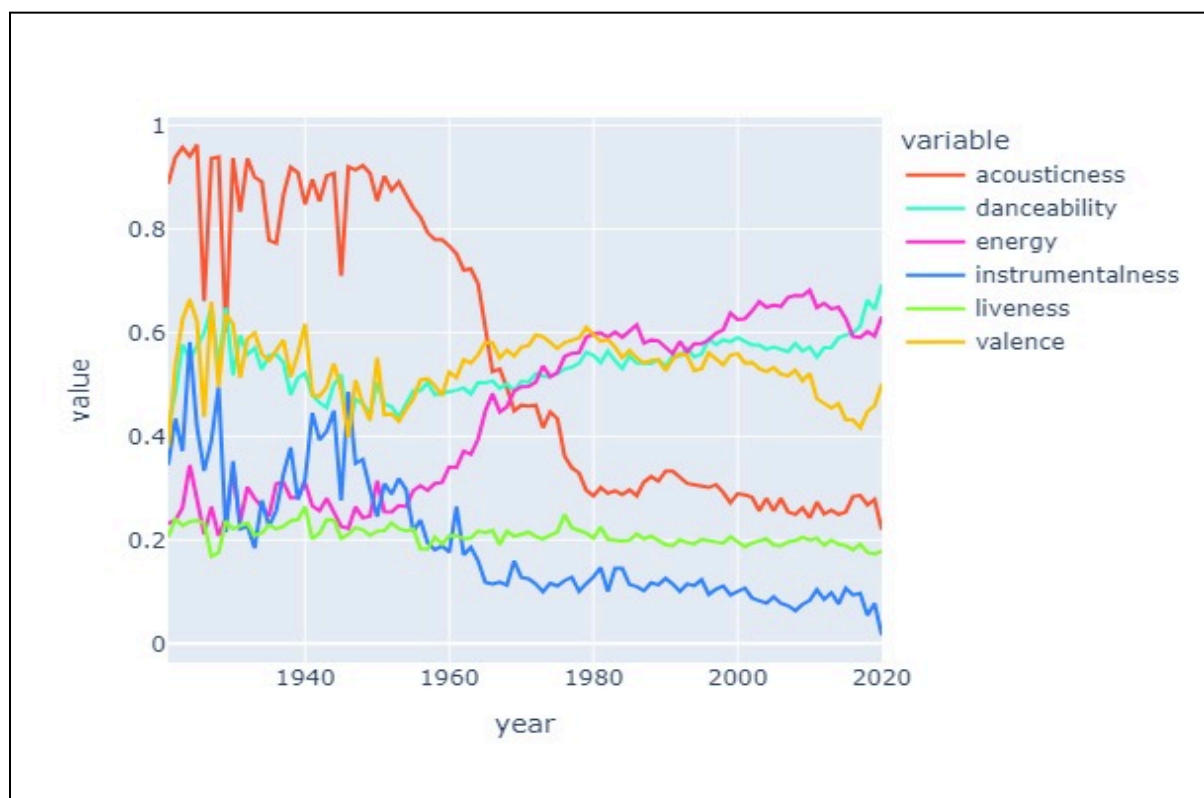




Features correlation with dependent variable



Music Over Time



Year to Feature Graph